Part A: Preprocessing the data

We begin by loading the data into `R` and taking a peek at the first few observations of the data set.

```
# Loading the data
df_raw <- read.csv("S1Data.csv")
```

```
# Taking a quick glimpse at the data
head(df_raw)
```

```
##   TIME Event Gender Smoking Diabetes BP Anaemia Age Ejection.Fraction Sodium
## 1   97     0      0       0        0  0       1  43                50    135
## 2  180     0      1       1        1  0       1  73                30    142
## 3   31     1      1       1        0  1       0  70                20    134
## 4   87     0      1       0        0  0       1  65                25    141
## 5  113     0      1       0        0  0       0  64                60    137
## 6   10     1      1       0        0  0       1  75                15    137
##   Creatinine Pletelets  CPK
## 1       1.30    237000  358
## 2       1.18    160000  231
## 3       1.83    263358  582
## 4       1.10    298000  305
## 5       1.00    242000 1610
## 6       1.20    127000  246
```

A rudimentary description of each variable is given below:

- `TIME`: Follow-up period (in days)
- `Event`: Whether death occurred during follow-up (1 = 'died', 0 = 'survived')
- `Gender`: Sex of the patient (1 = 'male', 0 = 'female')
- `Smoking`: Whether the patient smokes (1 = 'smoker', 0 = 'non-smoker')
- `Diabetes`: Whether the patient has diabetes (1 = 'diabetic', 0 = 'non-diabetic')
- `BP`: Whether the patient has hypertension (1 = 'hypertensive', 0 = 'non-hypertensive')
- `Anaemia`: Whether the patient has anaemia (1 = 'anaemic', 0 = 'non-anaemic')
- `Age`: Age of the patient (in years)
- `Ejection.Fraction`: The amount of blood exiting the heart per contraction (as a percentage)
- `Sodium`: The level of sodium in the blood (in mEq/L)
- `Creatinine`: The level of creatinine in the blood (in mg/dL)
- `Pletelets`: Blood platelet count (in platelet/μL)
- `CPK`: The level of creatine phosphokinase (CPK) enzyme in the blood (in μg/L)

The European Society of Cardiology guidelines, published in 2021, provide three subgroups of heart failure based on ejection fraction [1]:

- Heart failure with preserved ejection fraction (HFpEF): $\geq 50\%$
- Heart failure with moderately reduced ejection fraction (HFmrEF): 41-49%
- Heart failure with reduced ejection fraction (HFrEF): $\leq 40\%$

We therefore create a new variable, namely `ef_grp`, corresponding to these clinically defined levels. All other variables are renamed for enhanced clarity and a copy of the raw data set is created for clean-up.

```r
# Aggregating variables
vars <- c("time", "event", "sex", "smoker", "diabetic", "high_bp", "anaemic",
          "age", "ef", "serum_s", "serum_c", "platelet", "cpk")
```

```r
# Creating new data frame for clean-up
df_clean <- df_raw

# Renaming column titles
names(df_clean) <- vars
```

```r
# Mutating the data frame to include ejection fraction categories
df_clean$ef_grp <- cut(df_clean$ef, breaks = c(-Inf, 40, 49, Inf),
                       labels = c("HFrEF", "HFmrEF", "HFpEF"))
```

In order to aid us in interpretation, a dictionary is created to briefly explain how each variable has been measured.

```r
# Creating a dictionary to explain each variable
df_dict <- data.frame(
  Variable = c(vars, "ef_group"),
  Description = c(
    paste("Follow-up period"),
    paste("Whether death occurred during follow-up"),
    paste("Sex of the patient"),
    paste("Whether the patient currently smokes"),
    paste("Whether the patient has diabetes"),
    paste("Whether the patient has hypertension"),
    paste("Whether the patient has anaemia"),
    paste("Age of the patient"),
    paste("Ejection fraction (the amount of blood exiting the heart per contraction)"),
    paste("Serum sodium (the level of sodium in the blood)"),
    paste("Serum creatinine (the level of creatinine in the blood)"),
    paste("Blood platelet count"),
    paste("The level of creatine phosphokinase (CPK) enzyme in the blood)"),
    paste("Heart failure classification based on ejection fraction")
  ),
  Units = c(
    paste("Days"),
    paste("Binary (1 = died, 0 = survived)"),
    paste("Binary (1 = male, 0 = female)"),
    paste("Binary (1 = yes, 0 = no)"),
    paste("Binary (1 = yes, 0 = no)"),
    paste("Binary (1 = yes, 0 = no)"),
    paste("Binary (1 = yes, 0 = no)"),
    paste("Years"),
    paste("Percentage"),
    paste("Milliequivalents per litre (mEq/L)"),
    paste("Milligrams per decilitre (mg/dL)"),
    paste("Platelets per microlitre (platelet/mcL)"),
    paste("Micrograms per litre (mcg/L)"),
    paste("Ordinal (1 = HFpEF, 2 = HFmrEF, 3 = HFrEF)")
  ),
```

```
  stringsAsFactors = FALSE
)
```

```
# Viewing the dictionary in R (drag the header cell borders to change the column width)
View(df_dict)
```

Let us now see if the data needs to be cleaned; we start by checking for any missing values.

```
# Checking for missing values
colSums(is.na(df_clean))
```

```
##     time    event      sex   smoker diabetic  high_bp  anaemic      age
##        0        0        0        0        0        0        0        0
##       ef  serum_s  serum_c platelet      cpk   ef_grp
##        0        0        0        0        0        0
```

The output above tells us, succinctly, that there are no missing entries to delete or impute. Furthermore, we inspect the internal structure of our data frame; in particular, the *storage mode* (type of data) of each variable is of interest.

```
# Checking the structure of our data frame
str(df_clean)
```

```
## 'data.frame':    299 obs. of  14 variables:
##  $ time    : int  97 180 31 87 113 10 250 27 87 87 ...
##  $ event   : int  0 0 1 0 0 1 0 1 0 0 ...
##  $ sex     : int  0 1 1 1 1 1 1 1 1 1 ...
##  $ smoker  : int  0 1 1 0 0 0 1 0 0 1 ...
##  $ diabetic: int  0 1 0 0 0 0 0 1 0 0 ...
##  $ high_bp : int  0 0 1 0 0 0 0 1 1 0 ...
##  $ anaemic : int  1 1 0 1 0 1 0 0 0 0 ...
##  $ age     : num  43 73 70 65 64 75 70 94 75 80 ...
##  $ ef      : int  50 30 20 25 60 15 40 38 45 25 ...
##  $ serum_s : int  135 142 134 141 137 137 136 134 137 144 ...
##  $ serum_c : num  1.3 1.18 1.83 1.1 1 1.2 2.7 1.83 1.18 1.1 ...
##  $ platelet: num  237000 160000 263358 298000 242000 ...
##  $ cpk     : int  358 231 582 305 1610 246 582 582 582 898 ...
##  $ ef_grp  : Factor w/ 3 levels "HFrEF","HFmrEF",..: 3 1 1 1 3 1 1 1 2 1 ...
```

We would expect the variable `age` to be of storage mode `int` (age, in years, takes integer values). Let us see what the fuss is about.

```
# Checking observations with non-integer ages
subset(df_clean, age %% 1 != 0)
```

```
##     time event sex smoker diabetic high_bp anaemic    age ef serum_s serum_c
## 136  171     1   1      0        1       0       1 60.667 30     136     1.5
## 288  172     0   0      0        1       1       1 60.667 40     136     1.0
##     platelet cpk ef_grp
## 136   389000 104  HFrEF
## 288   201000 151  HFrEF
```

Oddly enough, there are two observations with non-integer ages. We expect this to bear such negligible influence on our results. Nonetheless, for completeness, we round these ages down using the `floor()` function (those who are, mathematically speaking, 60.667-years-old would, most probably, state their age as 60-years-old in reality).

```
# Rounding ages
df_clean$age <- floor(df_clean$age)
```

Finally, we convert the categorical variables into factors.

```
df_clean$event <- factor(df_clean$event, levels = c(0, 1),
                         labels = c("Survived", "Died"))
df_clean$sex <- factor(df_clean$sex, levels = c(0, 1),
                       labels = c("Female", "Male"))
df_clean$smoker <- factor(df_clean$smoker,levels = c(0, 1),
                          labels = c("Non-smoker", "Smoker"))
df_clean$diabetic <- factor(df_clean$diabetic, levels = c(0, 1),
                            labels = c("Non-diabetic", "Diabetic"))
df_clean$high_bp <- factor(df_clean$high_bp, levels = c(0, 1),
                           labels = c("No hypertension", "Hypertension"))
df_clean$anaemic <- factor(df_clean$anaemic, levels = c(0, 1),
                           labels = c("Non-anaemic", "Anaemic"))
```

Part B: Exploratory data analysis (EDA)

Out of the 299 observations, 96 ($\approx 32\%$) patients died and 203 ($\approx 68\%$) patients survived. We deduce this easily using the `table()` function.

```
table(df_clean$event)
```

```
##
## Survived     Died
##      203       96
```

Given the variable **event** as our proposed regressor, this represents an imbalanced data set, with over twice as many survivors as non-survivors. We discuss the implications of this later.

Let us continue by partitioning the variables into their continuous and categorical constituents. We exclude both **time** and **event** here due to the nature of our analyses.

```
# Aggregating continuous and categorical variables separately
cont_vars <- c("age", "ef", "serum_s", "serum_c", "platelet", "cpk")
cat_vars <- c("sex", "smoker", "diabetic", "high_bp", "anaemic", "ef_grp")
```

```
# Summary statistics for the continuous variables
summary(df_clean[cont_vars])
```

```
##       age              ef           serum_s         serum_c
##  Min.   :40.00   Min.   :14.00   Min.   :113.0   Min.   :0.500
##  1st Qu.:51.00   1st Qu.:30.00   1st Qu.:134.0   1st Qu.:0.900
```

```
##  Median :60.00   Median :38.00   Median :137.0   Median :1.100
##  Mean   :60.83   Mean   :38.08   Mean   :136.6   Mean   :1.394
##  3rd Qu.:70.00   3rd Qu.:45.00   3rd Qu.:140.0   3rd Qu.:1.400
##  Max.   :95.00   Max.   :80.00   Max.   :148.0   Max.   :9.400
##     platelet           cpk
##  Min.   : 25100   Min.   :  23.0
##  1st Qu.:212500   1st Qu.: 116.5
##  Median :262000   Median : 250.0
##  Mean   :263358   Mean   : 581.8
##  3rd Qu.:303500   3rd Qu.: 582.0
##  Max.   :850000   Max.   :7861.0
```

```r
# Histogram plotting function
plot_histogram <- function(varname, data = df_clean, xlab_text = NULL) {
  x <- data[[varname]]

  # If no custom x-axis label provided, use variable name
  if (is.null(xlab_text)) {
    xlab_text <- varname
  }

  # Base histogram
  hist(x,
       breaks = 30,
       col = "skyblue3",
       border = "white",
       main = paste("Distribution of", varname),
       xlab = xlab_text,
       ylab = "Frequency",
       freq = FALSE)

  # Overlaying density curve
  lines(density(x), col = "firebrick3", lwd = 2)

  # Adding data rug
  rug(x, col = "gray50")
}

# Example: plot_histogram("age", xlab_text = "Age (years)")
```

See Appendix B1 for the histogram/distribution plot of each continuous variable.

A predominantly older cohort of patients was observed, with ages ranging from 40 to 95 (mean $\approx$ 61). Ejection fraction values spanned 14-80%, with a mean of approximately 38%, suggesting that most patients had reduced systolic function.

Serum sodium levels averaged around 136 mEq/dL (a normal range is considered to be between 133-146 mEq/dL [2]), showing preserved electrolyte balance; serum creatinine levels, however, averaged around 1.39 mg/dL, ranging up to 9.4 mg/dL, highlighting a subset of patients with *renal* (kidney) impairment [3].

Blood platelet counts varied widely (25100-850000 platelets/μL), clustering around 263000 platelets/μL (consistent with healthy levels [4]). In contrast, creatine phosphokinase (CPK) levels exhibited extreme right-skew, with a median of 250 μg/L, a mean of around 580 μg/L and a maximum value of around 7800 μg/L.

We then create boxplots comparing the distribution of continuous variables between survivors and non-survivors.

```
# Boxplot function for continuous variables grouped by death event
plot_boxplot <- function(varname, data = df_clean, ylab_text = NULL) {
  x <- data[[varname]]
  outcome <- data$event

  if (is.null(ylab_text)) {
    ylab_text <- varname
  }

  boxplot(x ~ outcome,
          col = c("skyblue3", "firebrick3"),
          border = "gray30",
          ylab = ylab_text,
          xlab = "",
          main = paste("Distribution of", varname, "by death event"))
}

# Example: plot_boxplot("age", ylab_text = "Age (years)")
```

The figure in Appendix B2 tells us that non-survivors were generally older and had markedly lower ejection fractions; they also had slightly lower serum sodium levels and slightly higher serum creatinine levels.

Stacked barplots are formed (as seen in Appendix B3) to illustrate the survival and death proportions across each categorical variable.

```
# Barplot function for categorical variables
plot_barplot <- function(varname, data = df_clean) {
  # Contingency and percentage tables
  tbl <- table(data[[varname]], data$event)
  prop_tbl <- prop.table(tbl, 1) * 100

  # Creating stacked barplots
  mids <- barplot(t(prop_tbl),
                  col = c("skyblue3", "firebrick3"),
                  ylim = c(0, 100),
                  ylab = "Percentage within group",
                  xlab = "",
                  main = paste("Death outcome by", varname))

  # Adding percentage labels
  text(mids,
       prop_tbl[, "Survived"] / 2,
       paste0(round(prop_tbl[, "Survived"], 1), "% survived"),
       col = "white", cex = 0.8)
  text(mids,
       prop_tbl[, "Survived"] + prop_tbl[, "Died"] / 2,
       paste0(round(prop_tbl[, "Died"], 1), "% died"),
       col = "white", cex = 0.8)
}

# Example: plot_barplot("sex")
```

We note that mortality rates were slightly higher amongst anaemic and hypertensive patients, as well as those with a reduced ejection fraction (HFrEF). By contrast, sex, smoking and diabetes status showed little difference in survival proportions between groups.

Now, we produce a correlation matrix (comparing only our numeric variables).

```
cont_data <- df_clean[cont_vars]

cor_matrix <- cor(cont_data, method = "pearson")
cor_matrix
```

```
##                  age          ef     serum_s     serum_c    platelet
## age       1.00000000  0.06019547 -0.04591178  0.15923697 -0.05247529
## ef        0.06019547  1.00000000  0.17590228 -0.01130247  0.07217747
## serum_s  -0.04591178  0.17590228  1.00000000 -0.18909521  0.06212462
## serum_c   0.15923697 -0.01130247 -0.18909521  1.00000000 -0.04119808
## platelet -0.05247529  0.07217747  0.06212462 -0.04119808  1.00000000
## cpk      -0.08140639 -0.04407955  0.05955016 -0.01640848  0.02446339
##                 cpk
## age      -0.08140639
## ef       -0.04407955
## serum_s   0.05955016
## serum_c  -0.01640848
## platelet  0.02446339
## cpk       1.00000000
```

The output above reveals that the continuous predictors are only very weakly correlated with one another. A visual representation thus seems redundant as we no longer comment on the risk of multicollinearity.

To summarise, mortality was primarily associated with older age, reduced ejection fraction and elevated renal injury markers. Categorical factors, such as anaemia and hypertension, also corresponded to higher death rates. Weak intercorrelations suggest that these predictors offer distinct information for modelling.

Part C: Modelling

In this classification problem, we scrutinise the use of logistic regression as a method to predict whether a patient dies during the follow-up period based on a variety of predictors.

```
# Defining appropriate predictors (excluding 'ef_grp' due to multicollinearity)
preds <- setdiff(names(df_clean), c("event", "time", "ef_grp"))

# Setting reference level for death event/outcome
df_clean$event <- relevel(df_clean$event, ref = "Survived")

# Omitting 'ef_grp' in our cleaned data
df_clean$ef_grp <- NULL

# Creating model matrix
X <- model.matrix(event ~ ., data = df_clean[, c(preds, "event")])[ , -1]
y <- df_clean$event
```

Here, we use the `caret` package to partition the data into training and test sets (80%/20% split). Stratified sampling is automatically employed to preserve class balance [5].

```r
set.seed(123) # Setting seed for reproducible results

# Splitting into stratified 80/20 train/test sets
n <- nrow(df_clean)
train_index <- createDataPartition(y, p = 0.8, list = FALSE)

X_train <- X[train_index, ]
X_test  <- X[-train_index, ]
y_train <- y[train_index]
y_test  <- y[-train_index]
```

Next, we normalise our continuous variables via standardisation. The process is as follows:

1. Calculate the (distribution) mean and standard deviation for each predictor
2. Subtract the mean from each value
3. Divide the result by the corresponding standard deviation

Mathematically,

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j},$$

where

- x_{ij} is the original value of the j-th predictor for the i-th observation
- μ_j is the mean of the j-th predictor
- σ_j is the standard deviation of the j-th predictor
- z_{ij} is the standardised (normalised) value of x_{ij}

This decision is paramount when adopting regularisation methods, as penalties are sensitive to feature scaling [6]. A more involved explanation is included in the individual reflection.

```r
# Computing mean and sd from training data
train_means <- apply(X_train[, cont_vars], 2, mean)
train_sds <- apply(X_train[, cont_vars], 2, sd)

# Scaling training data
X_train[, cont_vars] <- scale(X_train[, cont_vars],
                              center = train_means,
                              scale = train_sds)

# Applying same scaling to test data
X_test[, cont_vars] <- scale(X_test[, cont_vars],
                             center = train_means,
                             scale = train_sds)

train_df <- data.frame(y_train, X_train)
```

We now fit the following logistic regression model:

$$\text{logit}\left(\mathbb{P}(Y_i = 1 \mid \mathbf{x}_i)\right) = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_{11} x_{i11},$$

where

- $Y_i \in \{0, 1\}$ is the (binary) death outcome for observation i
- $\mathbf{x}_i = (x_{i1}, \ldots, x_{i11})$ is a vector of all (11) predictors
- $\beta_0, \ldots, \beta_{13}$ are the corresponding model coefficients estimated by maximum likelihood
- the model assumes $Y_i \sim \text{Bernoulli}(p_i)$, with $p_i = \mathbb{P}(Y_i = 1 \mid \mathbf{x}_i)$

The logit function is defined by the equation

$$\text{logit}(p) = \ln\left(\frac{p}{1-p}\right)$$

and the inverse relationship gives predicted probabilities as

$$\begin{aligned}\mathbb{P}(Y_i = 1 \mid \mathbf{x}_i) &= \text{logit}^{-1}\left(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_{13} x_{i13}\right) \\ &= \frac{1}{1 + \exp(-\beta_0 + \beta_1 x_{i1} + \cdots + \beta_{13} x_{i13})}\end{aligned}$$

```r
# Fitting basic logistic regression model
full_model <- glm(y_train ~ ., data = train_df,
                  family = binomial(link = "logit"))

summary(full_model)
```

```
##
## Call:
## glm(formula = y_train ~ ., family = binomial(link = "logit"),
##     data = train_df)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -2.6411  -0.7429  -0.4326   0.7626   2.4912
##
## Coefficients:
##                     Estimate Std. Error z value Pr(>|z|)
## (Intercept)         -1.00892    0.40149  -2.513 0.011974 *
## sexMale             -0.52938    0.40215  -1.316 0.188050
## smokerSmoker         0.31562    0.38956   0.810 0.417835
## diabeticDiabetic     0.16309    0.33897   0.481 0.630410
## high_bpHypertension  0.23604    0.35380   0.667 0.504672
## anaemicAnaemic       0.24169    0.33974   0.711 0.476842
## age                  0.61722    0.17995   3.430 0.000604 ***
## ef                  -0.96570    0.21148  -4.566 4.96e-06 ***
## serum_s             -0.16246    0.16865  -0.963 0.335378
## serum_c              0.86372    0.23900   3.614 0.000302 ***
## platelet            -0.08092    0.18047  -0.448 0.653871
## cpk                  0.29975    0.16230   1.847 0.064771 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 301.20  on 239  degrees of freedom
## Residual deviance: 229.81  on 228  degrees of freedom
## AIC: 253.81
##
## Number of Fisher Scoring iterations: 5
```

The variables `age`, `ef` and `serum_c` are all significant here. Model convergence is achieved after five Fisher scoring iterations; the model's goodness-of-fit is indicated by the null and residual deviances (301.20 and 229.81, respectively) and an AIC of 253.81.

A patient who smokes and/or has diabetes is at risk of hypertension [7, 8]. Thus, we slightly bulk the previous model with two interaction terms built on prior information:

$$\begin{aligned}\text{logit}\left(\mathbb{P}(Y_i = 1 \mid \mathbf{x}_i)\right) &= \beta_0 + \beta_1 x_{i1} + \ldots \\ &\quad + \beta_{11} x_{i11} + \beta_{12}\left(\texttt{smoker}_i \times \texttt{hypertentive}_i\right) + \beta_{13}\left(\texttt{diabetic}_i \times \texttt{hypertentive}_i\right),\end{aligned}$$

where

- $\texttt{smoker}_i$, $\texttt{diabetic}_i$ and $\texttt{hypertentive}_i$ are (binary) indicator variables for smoking, diabetes and high blood pressure, respectively
- $\texttt{smoker}_i \times \texttt{hypertentive}_i$ and $\texttt{diabetic}_i \times \texttt{hypertentive}_i$ denote the interaction terms

```r
# Adding interaction terms to the logistic regression model
int_model <- glm(
  y_train ~ . + smokerSmoker:high_bpHypertension + diabeticDiabetic:high_bpHypertension,
  data = train_df,
  family = binomial(link = "logit")
)

summary(int_model)
```

```
##
## Call:
## glm(formula = y_train ~ . + smokerSmoker:high_bpHypertension +
##     diabeticDiabetic:high_bpHypertension, family = binomial(link = "logit"),
##     data = train_df)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -2.6868  -0.7370  -0.4176   0.7418   2.5842
##
## Coefficients:
##                                       Estimate Std. Error z value Pr(>|z|)
## (Intercept)                            -0.8500     0.4216  -2.016 0.043809 *
## sexMale                                -0.6087     0.4066  -1.497 0.134303
## smokerSmoker                           -0.1419     0.4665  -0.304 0.760935
## diabeticDiabetic                        0.2142     0.4231   0.506 0.612618
## high_bpHypertension                    -0.1046     0.5120  -0.204 0.838053
## anaemicAnaemic                          0.2729     0.3445   0.792 0.428392
## age                                     0.6275     0.1831   3.427 0.000610 ***
## ef                                     -1.0005     0.2128  -4.702 2.58e-06 ***
## serum_s                                -0.1859     0.1695  -1.097 0.272620
## serum_c                                 0.8355     0.2438   3.427 0.000611 ***
## platelet                               -0.1050     0.1861  -0.564 0.572680
## cpk                                     0.3228     0.1658   1.946 0.051648 .
## smokerSmoker:high_bpHypertension        1.4824     0.7640   1.940 0.052341 .
## diabeticDiabetic:high_bpHypertension   -0.2725     0.7059  -0.386 0.699426
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 301.20  on 239  degrees of freedom
## Residual deviance: 225.88  on 226  degrees of freedom
## AIC: 253.88
##
## Number of Fisher Scoring iterations: 5
```

Note that `cpk` and `smokerSmoker:high_bpHypertension` are teetering on the edge of statistical significance. Compared to the previous model, the difference in results is marginal.

An analysis of deviance (ANOVA) test is employed to compare the two nested models. The `full_model` is nested within `int_model`, as the latter includes all predictors from the former along with the additional interaction terms.

Let $\mathcal{M}_1$ denote the simpler main-effects model and $\mathcal{M}_2$ denote the extended interaction model. The inclusion of both interaction terms is assessed by comparing the log-likelihoods of both models:

$$\mathcal{D} = -2\left(\ell_{\mathcal{M}_1} - \ell_{\mathcal{M}_2}\right),$$

where $\ell_{\mathcal{M}_1}$ and $\ell_{\mathcal{M}_2}$ are the maximised log-likelihoods of `full_model` and `int_model`, respectively.

The test evaluates whether the additional interaction effects significantly improve model fit:

$$H_0 : \beta_{12} = \beta_{13} = 0$$
$$H_1 : \exists\ j \in \{12, 13\} \implies \beta_j \neq 0$$

Under the null hypothesis, the statistic $\mathcal{D}$ approximately follows a χ^2_k distribution, with k equal to the difference in model degrees-of-freedom.

```r
# Analysis of deviance test
anova(full_model, int_model, test = "Chisq")
```

```
## Analysis of Deviance Table
##
## Model 1: y_train ~ sexMale + smokerSmoker + diabeticDiabetic + high_bpHypertension +
##     anaemicAnaemic + age + ef + serum_s + serum_c + platelet +
##     cpk
## Model 2: y_train ~ sexMale + smokerSmoker + diabeticDiabetic + high_bpHypertension +
##     anaemicAnaemic + age + ef + serum_s + serum_c + platelet +
##     cpk + smokerSmoker:high_bpHypertension + diabeticDiabetic:high_bpHypertension
##   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
## 1       228     229.81
## 2       226     225.88  2   3.9211   0.1408
```

Given a p-value of $p = 0.1408$, there is insufficient evidence to reject H_0. From this, we infer that there is no statistically significant evidence that including the interaction terms improves the model's explanatory power.

Bidirectional stepwise selection seeks the model that minimises the Akaike Information Criterion (AIC), defined as

$$\text{AIC}(\mathcal{M}) := -2\ell(\hat{\beta}) + 2k,$$

where $\ell(\hat{\beta})$ is the maximised log-likelihood of model $\mathcal{M}$ and k is the number of estimated parameters.

The algorithm iteratively evaluates the inclusion or exclusion of predictors based on their contribution to the AIC:

- Forward selection: includes variables that most reduce the AIC
- Backward selection: excludes variables whose exclusion most reduces the AIC

At every iteration, both directions are considered; the process continues until no further addition or removal of predictors yields a lower AIC value.

```
# Bidirectional stepwise model selection
step_full <- stepAIC(full_model, direction = "both", trace = FALSE)

summary(step_full)
```

```
##
## Call:
## glm(formula = y_train ~ age + ef + serum_c + cpk, family = binomial(link = "logit"),
##     data = train_df)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -2.5621  -0.7465  -0.4872   0.8232   2.5131
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -0.9822     0.1774  -5.536 3.10e-08 ***
## age           0.5865     0.1731   3.387 0.000706 ***
## ef           -0.9652     0.2072  -4.659 3.17e-06 ***
## serum_c       0.8593     0.2306   3.726 0.000195 ***
## cpk           0.2247     0.1512   1.487 0.137135
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 301.20  on 239  degrees of freedom
## Residual deviance: 234.16  on 235  degrees of freedom
## AIC: 244.16
##
## Number of Fisher Scoring iterations: 5
```

The final selected model includes age, ejection fraction, serum creatinine and CPK levels. On the basis of AIC alone, we would favour this model over the standard logistic regression model. Bar CPK levels, these were also the predictors identified as significant in the initial model.

Again, we perform an analysis of deviance test to compare `full_model` with `step_model`.

```
# Analysis of deviance test
anova(step_full, full_model, test = "Chisq")
```

```
## Analysis of Deviance Table
##
## Model 1: y_train ~ age + ef + serum_c + cpk
## Model 2: y_train ~ sexMale + smokerSmoker + diabeticDiabetic + high_bpHypertension +
##     anaemicAnaemic + age + ef + serum_s + serum_c + platelet +
##     cpk
```

```
##   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
## 1       235     234.16
## 2       228     229.81  7   4.3533   0.7383
```

With $p = 0.7383$, we, again, favour the initial model.

We now bring regularisation (or penalisation – both words are used interchangeably here) into the mix. *Elastic net* is a form of regularised regression that combines:

- L_1 penalty (LASSO): encourages sparsity by shrinking coefficients *to* zero (effectively performing feature selection)
- L_2 penalty (ridge): shrinks coefficients smoothly *towards* zero

The elastic net loss function is given in the following form:

$$\text{Loss}(\alpha, \lambda) = \sum_{i=1}^{n}(y_i - \hat{y}_i) + \lambda \left(\alpha \sum_{j=1}^{p} |\beta_j| + (1-\alpha) \sum_{j=1}^{p} \beta_j^2 \right),$$

where

- n is the number of observations and p is the number of predictors
- λ controls the overall strength of the penalisation
- $\alpha \in [0, 1]$ determines the mix between L_1 and L_2 penalisation, e.g. when $\alpha = 1$, the problem collapses into simple ridge regression

Below, we run a nested search over candidate values of α in order to find the corresponding optimal value of λ, resulting in a tuned combination of hyperparameters that maximises the cross-validated Area Under Curve (AUC) value.

In cross-validation, we split the data into K folds. For each fold $1, \ldots, K$:

1. Train the model on the remaining $K - 1$ folds
2. Predict on the held-out fold
3. Compute the AUC

Then, we average over all folds:

$$\text{CV-AUC}(\alpha, \lambda) = \frac{1}{K} \sum_{k=1}^{K} \text{AUC}_k(\alpha, \lambda).$$

Finally, we select the hyperparameters that generalise best to unseen data:

$$(\alpha^*, \lambda^*) = \arg\max \text{CV-AUC}(\alpha, \lambda).$$

```r
# Converting regressor to binary for glmnet
y_train_bin <- ifelse(y_train == "Died", 1, 0)
y_test_bin  <- ifelse(y_test == "Died", 1, 0)
```

```r
# Elastic net cross-validation using AUC as the metric
alphas <- seq(0, 1, by = 0.1)
cv_results <- data.frame(alpha = numeric(), lambda = numeric(), auc = numeric())
```

```r
set.seed(123)

for (a in alphas) {
  # 10-fold CV
  cv_fit <- cv.glmnet(as.matrix(X_train), y_train_bin, family = "binomial",
                      alpha = a, type.measure = "auc", nfolds = 10)

  # Identifying best lambda (highest mean AUC)
  opt_idx <- which.max(cv_fit$cvm)

  # Storing alpha, lambda and best AUC score
  cv_results <- rbind(cv_results, data.frame(
    alpha = a,
    lambda = cv_fit$lambda[opt_idx],
    auc = max(cv_fit$cvm)
  ))

  # Printing progress
  cat(sprintf("Alpha = %.1f | Optimal lambda = %.5f | CV AUC = %.5f\n",
              a, cv_fit$lambda[opt_idx], max(cv_fit$cvm)))
}
```

```
## Alpha = 0.0 | Optimal lambda = 0.11026 | CV AUC = 0.79524
## Alpha = 0.1 | Optimal lambda = 0.51178 | CV AUC = 0.79900
## Alpha = 0.2 | Optimal lambda = 0.14643 | CV AUC = 0.80397
## Alpha = 0.3 | Optimal lambda = 0.14163 | CV AUC = 0.80035
## Alpha = 0.4 | Optimal lambda = 0.06671 | CV AUC = 0.82440
## Alpha = 0.5 | Optimal lambda = 0.17887 | CV AUC = 0.78424
## Alpha = 0.6 | Optimal lambda = 0.12375 | CV AUC = 0.77799
## Alpha = 0.7 | Optimal lambda = 0.04592 | CV AUC = 0.81121
## Alpha = 0.8 | Optimal lambda = 0.07705 | CV AUC = 0.80891
## Alpha = 0.9 | Optimal lambda = 0.11969 | CV AUC = 0.80065
## Alpha = 1.0 | Optimal lambda = 0.10772 | CV AUC = 0.80264
```

```r
# Finding overall best alpha-lambda combination (highest AUC)
opt_pars <- cv_results[which.max(cv_results$auc), ]

print(opt_pars)
```

```
##   alpha     lambda       auc
## 5   0.4 0.06671013 0.8243978
```

```r
# Fitting final elastic net model using optimal alpha and lambda
elnet_model <- glmnet(as.matrix(X_train), y_train_bin, alpha = opt_pars$alpha,
                      lambda = opt_pars$lambda, family = "binomial")
```

Note that, as $\lambda << 1$, the penalty term contributes very little to the model. Nevertheless, after fitting the elastic net model, we trace out the Receiving Operating Characteristic (ROC) curve and calculate the corresponding AUC.

The predicted probabilities are compared against the true binary outcomes, $y_i \in \{0, 1\}$, across a range of

classification thresholds, $t \in [0, 1]$. For each threshold t,

$$y_i = \begin{cases} 1 & \text{if } \hat{p}_i \geq t \\ 0 & \text{if } \hat{p}_i < t, \end{cases}$$

where $\hat{p}_i \in (0, 1)$ is the predicted probability of the 'positive' outcome (in this case, morbidly, death). The true positive (TP) and false positive (FP) rates (TPR and FPR, respectively), based on a chosen threshold, are calculated as follows:

$$\text{TPR}(t) = \frac{\text{TP}(t)}{\text{TP}(t) + \text{FN}(t)}$$
$$\text{FPR}(t) = \frac{\text{FP}(t)}{\text{FP}(t) + \text{TN}(t)},$$

where

- FN/TN denotes false/true negatives

An ROC curve is plotted by adjusting the value of t:

$$\text{ROC} = \{(\text{FPR}(t), \text{TPR}(t) : t \in [0, 1]\}$$

The area under the curve is usually calculated as an integral via the trapezoidal rule [9]:

$$A = \int_{x=0}^{x=1} \text{TPR}(\text{FPR}^{-1}(x)) \, \mathrm{d}x,$$

where

- $\text{TPR}(t) : t \to y(x)$ (the y-axis is the true positive rate)
- $\text{FPR}(t) : t \to x$ (the x-axis is the false positive rate)

It is equivalent to the probability that, given one positive (death) and one negative (survival) instance, the classifier is able to discern between the two outcomes. An AUC value of 1 represents perfect discrimination; a value of 0.5 means that the classifier is no better than random guessing [10, 11].

```
# Predicted probabilities
pred_full  <- predict(full_model, newdata = data.frame(X_test), type = "response")
pred_elnet <- predict(elnet_model, newx = as.matrix(X_test), type = "response")
```

```
# Finding ROC curves for both models
roc_full  <- roc(y_test_bin, pred_full)
roc_elnet <- roc(y_test_bin, as.vector(pred_elnet))
```

```
# Computing AUC values for both models
auc_full  <- auc(roc_full)
auc_elnet <- auc(roc_elnet)
```

```
auc_full; auc_elnet
```

```
## Area under the curve: 0.7276
```

Actual \ Predicted	1	0
1	True Positive (TP)	False Negative (FN)
0	False Positive (FP)	True Negative (TN)

Table 1: Confusion matrix layout for binary classification.

```
## Area under the curve: 0.7171
```

This means that both models can correctly identify a (randomly chosen) death or survival around 72% of the time.

We continue with an arbitrary threshold value of $t = 0.5$. A confusion matrix for both models is then formed. A table is displayed below, detailing the four entries of a confusion matrix.

From this, the following metrics are deduced:

- Accuracy $= \frac{TP+TN}{TP+TN+FP+FN}$
- Sensitivity (Recall) $= \frac{TP}{TP+FN}$
- Specificity $= \frac{TN}{TN+FP}$
- Precision $= \frac{TP}{TP+FP}$
- $F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$

```
# Threshold at 0.5
class_full <- ifelse(pred_full  > 0.5, 1, 0)
class_elnet <- ifelse(pred_elnet > 0.5, 1, 0)

cm_full <- confusionMatrix(factor(class_full), factor(y_test_bin),
                           positive = "1")
cm_elnet <- confusionMatrix(factor(class_elnet), factor(y_test_bin),
                           positive = "1")
```

```
cm_full
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##          0 34 10
##          1  6  9
##
##                Accuracy : 0.7288
##                  95% CI : (0.5973, 0.8364)
##     No Information Rate : 0.678
##     P-Value [Acc > NIR] : 0.2459
##
##                   Kappa : 0.3426
##
##  Mcnemar's Test P-Value : 0.4533
##
##             Sensitivity : 0.4737
##             Specificity : 0.8500
##          Pos Pred Value : 0.6000
##          Neg Pred Value : 0.7727
```

```
##              Prevalence : 0.3220
##          Detection Rate : 0.1525
##    Detection Prevalence : 0.2542
##       Balanced Accuracy : 0.6618
##
##        'Positive' Class : 1
##
```

```
cm_elnet
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##          0 36 15
##          1  4  4
##
##                Accuracy : 0.678
##                  95% CI : (0.5436, 0.7938)
##     No Information Rate : 0.678
##     P-Value [Acc > NIR] : 0.56176
##
##                   Kappa : 0.1303
##
##  Mcnemar's Test P-Value : 0.02178
##
##             Sensitivity : 0.2105
##             Specificity : 0.9000
##          Pos Pred Value : 0.5000
##          Neg Pred Value : 0.7059
##              Prevalence : 0.3220
##          Detection Rate : 0.0678
##    Detection Prevalence : 0.1356
##       Balanced Accuracy : 0.5553
##
##        'Positive' Class : 1
##
```

A Cohen's kappa value of 0.3426 for the logistic regression model indicates merely a slight agreement above chance; a value of 0.1303 means that the discriminatory power of the penalised model is even worse. This means that both models are only slightly better at performing compared to random guessing [12].

```
results <- data.frame(
  Model = c("Logistic", "Elastic Net"),
  AUC = c(auc_full, auc_elnet),
  Accuracy = c(cm_full$overall["Accuracy"],
               cm_elnet$overall["Accuracy"]),
  Sensitivity = c(cm_full$byClass["Sensitivity"],
                  cm_elnet$byClass["Sensitivity"]),
  Specificity = c(cm_full$byClass["Specificity"],
                  cm_elnet$byClass["Specificity"]),
  F1 = c(F1_Score(class_full, y_test_bin),
         F1_Score(class_elnet, y_test_bin))
```

```
)

print(results)
```

```
##         Model       AUC  Accuracy Sensitivity Specificity        F1
## 1    Logistic 0.7276316 0.7288136   0.4736842        0.85 0.8095238
## 2 Elastic Net 0.7171053 0.6779661   0.2105263        0.90 0.7912088
```

Ultimately, the logistic regression model seems to outperform the elastic net model on virtually all fronts (excluding specificity).

We decide to address the imbalance in the proportion of death outcomes by, firstly, introducing weights. For $y_i \in \{0, 1\}$, let:

- $n_1 = \sum_i \mathbf{1}(y_i = 1)$ be the number of 'positive' outcomes (death)
- $n_0 = \sum_i \mathbf{1}(y_i = 0)$ be the number of 'negative' outcomes (survival)

For each observation, a weight, w_i, is sampled:

$$w_i = \begin{cases} \frac{n_0}{n_1} & \text{if } y_i = 1 \\ 1 & \text{if } y_i = 0, \end{cases}$$

As a result, the weighted sum of positive cases is equal to the number of negative cases and the data set, effectively, becomes balanced in total weight:

$$\sum_{i:y_i=1} w_i = n_1 \cdot \frac{n_0}{n_1} = n_0$$

Such weights are then factored into our two regression models.

```
# Weighting positives (deaths) proportionally to imbalance
wts <- ifelse(y_train_bin == 1,
              sum(y_train_bin==0)/sum(y_train_bin==1), 1)

# Weighted logistic regression
w_full <- glm(y_train ~ ., data = train_df, family = binomial(link = "logit"),
               weights = wts)

# Weighted elastic net
w_elnet <- glmnet(as.matrix(X_train), y_train_bin,
                  family="binomial", alpha = opt_pars$alpha,
                  lambda = opt_pars$lambda, weights = wts)
```

```
# Predicted probabilities
pred_w_full  <- predict(w_full, newdata = data.frame(X_test), type = "response")
pred_w_elnet <- predict(w_elnet, newx = as.matrix(X_test), type = "response")

# Calculating ROC curves for all models
roc_w_full  <- roc(y_test_bin, pred_w_full)
roc_w_elnet <- roc(y_test_bin, as.vector(pred_w_elnet))

# Computing AUC values for all models
auc_w_full  <- auc(roc_w_full)
auc_w_elnet <- auc(roc_w_elnet)
```

```
auc_w_full; auc_w_elnet
```

```
## Area under the curve: 0.7171
```

```
## Area under the curve: 0.7237
```

The AUC values of the weighted models closely match those of the preceding models.

Matthews correlation coefficient (MCC) is utilised here to find an optimal threshold value. This criterion is considered to be "more informative" over other evaluation metrics (e.g. F1-score, accuracy) as it harmoniously accounts for all four confusion matrix categories [13]. Again, we use it to address class imbalance. It is computed as follows:

$$\text{MCC} = \frac{\text{TP} \cdot \text{TN} - \text{FP} \cdot \text{FN}}{\sqrt{(\text{TP}+\text{FP})(\text{TP}+\text{FN})(\text{TN}+\text{FP})(\text{TN}+\text{FN})}}$$

```
# Defining helper function to calculate MCC
mcc_for_thresh <- function(thresh, probs, actual) {
  preds <- ifelse(probs >= thresh, 1, 0)

  # Confusion matrix elements
  TP <- sum(actual == 1 & preds == 1)
  TN <- sum(actual == 0 & preds == 0)
  FP <- sum(actual == 0 & preds == 1)
  FN <- sum(actual == 1 & preds == 0)

  # Denominator term
  denom <- (TP+FP)*(TP+FN)*(TN+FP)*(TN+FN)

  # Dealing with division by zero
  eps <- 1e-9
  mcc <- (TP*TN-FP*FN) / sqrt((TP+FP+eps)*(TP+FN+eps)*(TN+FP+eps)*(TN+FN+eps))
}
```

```
# Searching thresholds for logistic regression model
thresholds <- seq(0.1, 0.9, by = 0.01)

mcc_vals_full <- sapply(thresholds, mcc_for_thresh, probs = pred_w_full,
                        actual = y_test_bin)
opt_thresh_full <- thresholds[which.max(mcc_vals_full)]
opt_mcc_full <- max(mcc_vals_full)

cat(sprintf("Optimal threshold (logistic regression) = %.3f | MCC = %.3f\n",
            opt_thresh_full, opt_mcc_full))
```

```
## Optimal threshold (logistic regression) = 0.550 | MCC = 0.396
```

```
# Searching thresholds for elastic net model
mcc_vals_elnet <- sapply(thresholds, mcc_for_thresh, probs = pred_w_elnet,
                         actual = y_test_bin)
opt_thresh_elnet <- thresholds[which.max(mcc_vals_elnet)]
opt_mcc_elnet <- max(mcc_vals_elnet)
```

```r
cat(sprintf("Optimal threshold (elastic net) = %.3f | MCC = %.3f\n",
            opt_thresh_elnet, opt_mcc_elnet))
```

```
## Optimal threshold (elastic net) = 0.550 | MCC = 0.443
```

```r
# Threshold at deduced optimum
class_w_full  <- ifelse(pred_full > opt_thresh_full, 1, 0)
class_w_elnet <- ifelse(pred_elnet > opt_thresh_elnet, 1, 0)

cm_w_full <- confusionMatrix(factor(class_w_full), factor(y_test_bin), positive = "1")
cm_w_elnet <- confusionMatrix(factor(class_w_elnet), factor(y_test_bin), positive = "1")

w_results <- data.frame(
  Model = c("Weighted Logistic", "Weighted Elastic Net"),
  AUC = c(auc_w_full, auc_w_elnet),
  Accuracy = c(cm_w_full$overall["Accuracy"], cm_w_elnet$overall["Accuracy"]),
  Sensitivity = c(cm_w_full$byClass["Sensitivity"], cm_w_elnet$byClass["Sensitivity"]),
  Specificity = c(cm_w_full$byClass["Specificity"], cm_w_elnet$byClass["Specificity"]),
  F1 = c(F1_Score(class_w_full, y_test_bin), F1_Score(class_w_elnet, y_test_bin))
)

print(w_results)
```

```
##                  Model       AUC  Accuracy Sensitivity Specificity        F1
## 1    Weighted Logistic 0.7171053 0.7118644   0.4210526       0.850 0.8000000
## 2 Weighted Elastic Net 0.7236842 0.6779661   0.1578947       0.925 0.7956989
```

Apart from a decrease in sensitivity for the penalised model, there are no major differences in metric values compared to each model's predecessor. Due to a minute difference in AUC between both models, on the basis of interpretability, we choose the weighted logistic regression model as our model for evaluation and fit it using the entire data set.

```r
# Combining all preprocessed data
df_final <- data.frame(y = y, X)

y_bin_final <- ifelse(df_final$y == "Died", 1, 0)

# Standardising numeric predictors (mean = 0, sd = 1)
df_final[cont_vars] <- scale(df_final[cont_vars])

# Class weights to address imbalanced data
wts_final <- ifelse(y_bin_final == 1,
                    sum(y_bin_final == 0) / sum(y_bin_final == 1), 1)

# Fitting final weighted logistic regression
final_w_logit <- glm(y ~ ., data = df_final,
                     family = binomial(link = "logit"), weights = wts_final)
```

```
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
```

```r
summary(final_w_logit)
```

```
##
## Call:
## glm(formula = y ~ ., family = binomial(link = "logit"), data = df_final,
##     weights = wts_final)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -2.6636  -1.0512  -0.6422   0.8673   3.2001
##
## Coefficients:
##                     Estimate Std. Error z value Pr(>|z|)
## (Intercept)          -0.4279     0.2952  -1.450   0.1471
## sexMale              -0.3744     0.2839  -1.319   0.1872
## smokerSmoker          0.1314     0.2812   0.467   0.6403
## diabeticDiabetic      0.1605     0.2406   0.667   0.5046
## high_bpHypertension   0.3916     0.2486   1.575   0.1152
## anaemicAnaemic        0.4485     0.2449   1.831   0.0671 .
## age                   0.6498     0.1258   5.165 2.40e-07 ***
## ef                   -0.7756     0.1334  -5.815 6.08e-09 ***
## serum_s              -0.2728     0.1220  -2.237   0.0253 *
## serum_c               0.7429     0.1753   4.238 2.25e-05 ***
## platelet             -0.0505     0.1247  -0.405   0.6855
## cpk                   0.2808     0.1143   2.456   0.0140 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 562.84  on 298  degrees of freedom
## Residual deviance: 439.02  on 287  degrees of freedom
## AIC: 451.06
##
## Number of Fisher Scoring iterations: 5
```

Odds ratios and their 95% confidence intervals are calculated to help explain the estimated coefficients of the model (that are of significance); we use the standard deviation values to 'rescale' the given values into more meaningful, interpretable results.

```r
# Computing (rounded) odds ratios and confidence intervals
or_table <- exp(cbind(OR = coef(final_w_logit), confint(final_w_logit)))
```

```r
# Rounding to 3 decimal places
or_table <- round(or_table, 3)

# Showing only significant variables
sig_vars <- c("age", "ef", "serum_s", "serum_c", "cpk")
or_table <- or_table[sig_vars, ]

or_table
```

```
##            OR 2.5 % 97.5 %
```

```
## age     1.915 1.506  2.468
## ef      0.460 0.351  0.594
## serum_s 0.761 0.597  0.964
## serum_c 2.102 1.552  3.089
## cpk     1.324 1.071  1.691
```

```r
# Means and SDs
num_stats <- data.frame(
  Mean = sapply(df_clean[, sig_vars, drop = FALSE], mean, na.rm = TRUE),
  SD   = sapply(df_clean[, sig_vars, drop = FALSE], sd, na.rm = TRUE)
)

round(num_stats, 3)
```

```
##            Mean      SD
## age      60.829  11.895
## ef       38.084  11.835
## serum_s 136.625   4.412
## serum_c   1.394   1.035
## cpk     581.839 970.288
```

Holding all other covariates constant, according to the model,

- A 10-year increase in age is associated with ~92% higher odds of death (OR ≈ 1.92, $\sigma = 11.895 \approx 10$)
- A 10% increase in ejection fraction reduces the odds of death by ~54% (OR ≈ 0.46, $\sigma = 11.835 \approx 10$)
- Each 5 mEq/L increase in serum sodium reduces the odds of death by ~24% (OR ≈ 0.76, $\sigma = 4.412 \approx 5$)
- Each 1 mg/dL increase in serum creatinine more than doubles the odds of death (OR ≈ 2, $\sigma = 1.035 \approx 1$)
- For every 1000 μg/L increase in creatine phosphokinase, the odds of death increase by ~32% (OR ≈ 1.32, $\sigma = 970.228 \approx 1000$)

Part D: Model evaluation

To assess the performance and adequacy of our weighted logistic regression model, we carry out a series of diagnostic and validation tests, as outlined below.

```r
# Multicollinearity
vif(final_w_logit)
```

```
##             sexMale        smokerSmoker    diabeticDiabetic high_bpHypertension
##            1.367470            1.264617            1.054287            1.053325
##      anaemicAnaemic                 age                  ef             serum_s
##            1.099926            1.149803            1.122956            1.070135
##             serum_c            platelet                 cpk
##            1.067605            1.052290            1.119726
```

```r
# Linearity assumption
fit <- glm(y_bin_final ~ age + ef + serum_s + serum_c + cpk,
           family = binomial(link = "logit"), data = df_final)
```

```r
crPlots(fit)
```

As seen in Appendix D1, the above outputs five component-plus-residual (CR) plots for each significant (continuous) predictor of the weighted logistic regression model.

```r
# Hosmer-Lemeshow goodness-of-fit test
hoslem.test(y_bin_final, fitted(final_w_logit))
```

```
##
##  Hosmer and Lemeshow goodness of fit (GOF) test
##
## data:  y_bin_final, fitted(final_w_logit)
## X-squared = 36.53, df = 8, p-value = 1.404e-05
```

```r
# Computing deviance residuals and fitted probabilities
resid_dev <- residuals(final_w_logit, type = "deviance")
fitted_vals <- fitted(final_w_logit)
```

```r
# Binned residual plot
binnedplot(
  x = fitted_vals,
  y = resid_dev,
  xlab = "Predicted probability",
  ylab = "Average deviance residuals",
  main = "Binned residual plot: (Weighted) logistic regression"
)
```

Appendix D2 includes the binned residual plot of the (weighted) logistic regression model.

```r
# ROC and AUC
roc_final <- roc(y_bin_final, fitted(final_w_logit))
auc(roc_final)
```

```r
# ROC plot
plot(roc_final, col = "blue", lwd = 2,
     main = "ROC curve: (Weighted) logistic regression model")
```

The figure in Appendix D3 includes the binned residual plot of the (weighted) logistic regression model.

```r
# Calibration data frame for (weighted) logistic regression model
cal_logit_df <- data.frame(
  Predicted = fitted(final_w_logit),
  Actual = y_bin_final
)

# Calibration plot with CIs
ggplot(cal_logit_df, aes(x = Predicted, y = Actual)) +
  geom_point(alpha = 0.3, size = 1.2) +
  geom_smooth(
    method = "loess",
    se = TRUE,
```

```
    color = "red",
    fill = "pink",
    span = 0.8
  ) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey40") +
  labs(
    title = "Calibration plot: (Weighted) logistic regression model",
    x = "Estimated Probability",
    y = "Observed Outcome"
  ) +
  theme_minimal(base_size = 13) +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5)
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

See Appendix D4 for the calibration plot of the (weighted) logistic regression model.

```
# Bootstrapping AUC 1000 times
set.seed(123)

auc_boot <- replicate(1000, {
  idx <- sample(seq_len(nrow(df_final)), replace = TRUE)
  mod <- glm(y ~ ., data = df_final[idx, ], family = binomial(link = "logit"),
             weights = wts_final[idx])
  pred <- predict(mod, newdata = df_final[-idx, ], type = "response")
  roc_obj <- roc(y_bin_final[-idx], pred)

  auc(roc_obj)
})
```

```
mean(auc_boot) # average bootstrap AUC
```

```
## [1] 0.7533881
```

```
sd(auc_boot) # standard deviation
```

```
## [1] 0.04325377
```

```
quantile(auc_boot, c(0.025, 0.975)) # 95% CIs
```

```
##      2.5%     97.5%
## 0.6622242 0.8337563
```

To accommodate for any non-linear effects, we fit natural cubic spline terms to the significant (continuous) predictors.

```r
# Fitting natural cubic spline model on full data
library(splines)

final_w_logit_spline <- glm(
  y_bin_final ~ ns(age, 3) + ns(ef, 3) + ns(serum_s, 3) + ns(serum_c, 3) +
    ns(cpk, 3) + platelet + sexMale + smokerSmoker + diabeticDiabetic +
    high_bpHypertension + anaemicAnaemic,
  data = df_final,
  family = binomial(link = "logit"),
  weights = wts_final
)
```

```r
# Function to plot splines
plot_spline <- function(model, var) {
  pred_obj <- ggpredict(model, terms = var)
  plot(pred_obj) +
    geom_line(color = "blue", linewidth = 0.9) +
    geom_ribbon(aes(ymin = conf.low, ymax = conf.high),
                alpha = 0.25, fill = "lightblue") +
    labs(
      title = paste("Effect of", var),
      x = var,
      y = "Predicted Probability"
    ) +
    theme_minimal(base_size = 10) +
    theme(
      plot.title   = element_text(hjust = 0.5, face = "bold", size = 11),
      axis.title.x = element_text(size = 9),
      axis.title.y = element_text(size = 9),
      axis.text.x  = element_text(size = 8),
      axis.text.y  = element_text(size = 8)
    )
}

# Example: plot_spline(final_w_logit_spline, "age")
```

Appendix D5 includes a figure that displays five cubic splines, corresponding to each significant (continuous) covariate.

```r
summary(final_w_logit_spline)
```

```
##
## Call:
## glm(formula = y_bin_final ~ ns(age, 3) + ns(ef, 3) + ns(serum_s,
##     3) + ns(serum_c, 3) + ns(cpk, 3) + platelet + sexMale + smokerSmoker +
##     diabeticDiabetic + high_bpHypertension + anaemicAnaemic,
##     family = binomial(link = "logit"), data = df_final, weights = wts_final)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -2.6644  -0.8391  -0.5775   0.5602   3.2521
##
## Coefficients:
```

```
##                     Estimate Std. Error z value Pr(>|z|)
## (Intercept)          1.00483    2.13723   0.470  0.63825
## ns(age, 3)1         -0.31159    0.61840  -0.504  0.61435
## ns(age, 3)2          4.27566    1.56718   2.728  0.00637 **
## ns(age, 3)3          4.94308    1.03220   4.789 1.68e-06 ***
## ns(ef, 3)1          -3.14258    0.69711  -4.508 6.54e-06 ***
## ns(ef, 3)2          -9.79856    2.10292  -4.659 3.17e-06 ***
## ns(ef, 3)3          -2.12483    1.43971  -1.476  0.13998
## ns(serum_s, 3)1     -0.49432    0.91892  -0.538  0.59062
## ns(serum_s, 3)2      0.11549    3.39795   0.034  0.97289
## ns(serum_s, 3)3     -0.19004    1.16024  -0.164  0.86989
## ns(serum_c, 3)1      4.35575    1.05886   4.114 3.90e-05 ***
## ns(serum_c, 3)2      5.14730    1.76547   2.916  0.00355 **
## ns(serum_c, 3)3      2.94459    1.83791   1.602  0.10912
## ns(cpk, 3)1         -0.89720    1.12842  -0.795  0.42656
## ns(cpk, 3)2          3.32594    1.13487   2.931  0.00338 **
## ns(cpk, 3)3          4.45342    1.83977   2.421  0.01549 *
## platelet            -0.06939    0.13354  -0.520  0.60330
## sexMale             -0.52536    0.31068  -1.691  0.09084 .
## smokerSmoker         0.26935    0.30837   0.873  0.38241
## diabeticDiabetic     0.39245    0.27225   1.441  0.14945
## high_bpHypertension  0.58506    0.27888   2.098  0.03591 *
## anaemicAnaemic       0.41403    0.27204   1.522  0.12803
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 562.84  on 298  degrees of freedom
## Residual deviance: 384.97  on 277  degrees of freedom
## AIC: 418.54
##
## Number of Fisher Scoring iterations: 5
```

```
AIC(final_w_logit, final_w_logit_spline)
```

```
##                       df      AIC
## final_w_logit         12 451.0642
## final_w_logit_spline  22 418.5361
```

```
roc_spline <- roc(y_bin_final, fitted(final_w_logit_spline))
auc(roc_spline)
```

```
## Area under the curve: 0.8566
```

```
cal_spline_df <- data.frame(predicted = fitted(final_w_logit_spline),
                            observed = y_bin_final)

# Calibration plot with CIs
ggplot(cal_spline_df, aes(x = predicted, y = observed)) +
  geom_point(alpha = 0.3, size = 1.2) +
  geom_smooth(
```

```
    method = "loess",
    se = TRUE,
    color = "red",
    fill = "pink",
    span = 0.8
  ) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey40") +
  labs(
    title = "Calibration plot: (Weighted) cubic spline model",
    x = "Estimated Probability",
    y = "Observed Outcome"
  ) +
  theme_minimal(base_size = 13) +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5)
  )
```

See Appendix D6 for the calibration plot of the natural cubic spline model.

Part E: Results and discussion

The weighted logistic regression model was first evaluated on the test set using probability predictions at a range of classification thresholds. ROC analysis yielded an AUC of 0.7276, indicating moderate discrimination, i.e., the model correctly ranked a randomly chosen death above a randomly chosen survivor approximately 73% of the time. To account for class imbalance in the outcome, MCC values were computed across thresholds ranging from 0.10 to 0.80. The optimal threshold under MCC was 0.55, achieving an MCC of 0.396

Next, the elastic net model was tuned via 10-fold cross-validation in order to minimise binomial deviance. Similar results were produced. Its test-set AUC was 0.7171, only slightly lower than the weighted logistic regression. MCC optimisation, again, identified 0.55 as the best threshold. Initially, the MCC computation returned an undefined value (NaN) due to a zero term in the denominator, a known numerical issue in small or imbalanced samples; introducing a small stabilising constant $\epsilon > 0$ mitigated this issue and yielded a valid MCC of 0.443. Despite the slightly higher MCC, the elastic net model did not demonstrate consistently superior performance, nor did it improve calibration or discrimination in a way that would justify preferring it over the weighted logistic regression model. This suggests that the benefits of regularisation were limited in this dataset.

A series of diagnostic tools was then used to assess whether the assumptions of the (linear) logistic regression model were satisfied.

Variance inflation factors (VIFs) for all predictors were below 5, indicating no problematic multicollinearity. However, the component-plus-residual (CR) plots revealed substantial deviations from linearity for several predictors — particularly serum creatinine, which showed a distinctly curved relationship with the log-odds of death.

The Hosmer–Lemeshow goodness-of-fit test yielded a strongly significant p-value ($p = 1.4 \times 10^{-5}$), providing evidence of poor calibration. Binned residual plots further demonstrated that the model underestimated mortality risk across the board, with predominantly negative residuals. Collectively, these diagnostics indicated that the assumption of linearity-in-the-logit was violated and that a more flexible approach was required.

To further assess the stability of the weighted logistic regression model, we performed a non-parametric bootstrap of the AUC with 1,000 resamples. In each iteration, we drew a bootstrap sample with replacement

and evaluated its discrimination on the corresponding out-of-bag data. The average bootstrap AUC was 0.753, closely aligning with the original test-set AUC, indicating that the model's discrimination is not overly sensitive to the particular sample split. The bootstrap distribution exhibited a standard deviation of 0.043, and the 95% confidence interval (0.662–0.834) suggested moderate variability. This is unsurprising, given the relatively small sample size and class imbalance.

Given the clear non-linear patterns, a natural cubic spline logistic regression model was fitted for the five continuous predictors: age, EF, serum sodium, serum creatinine, and CPK (each with 3 degrees of freedom). Partial effect plots (generated with the `ggeffects` package) confirmed the diagnostics:

- Age: A shallow slope at younger ages that steepened as age increased.
- Ejection fraction: A sharp decline in risk from very low EF values that flattened thereafter.
- Serum sodium: Near-linear behaviour with only minimal deviation.
- Serum creatinine: Strong non-linearity with an initial steep rise in mortality probability that plateaued at higher values.
- CPK: Mild non-linearity, mainly concentrated at low values.

Model fit improved with the addition of splines; calibration plots showed slightly better agreement between observed and predicted probabilities, with the LOESS curve tracking the 45° line more closely and with slightly narrower confidence bands. Spline terms for age, EF and serum creatinine were statistically significant ($p < 0.01$), supporting their inclusion. Although no single spline model was selected as "best", the improvements in calibration and fit demonstrate the added flexibility gained by relaxing prior linearity assumptions.

References

[1] Theresa A McDonagh, Marco Metra, Marianna Adamo, Roy S Gardner, Andreas Baumbach, Michael Böhm, Haran Burri, Javed Butler, Jelena Čelutkienė, Ovidiu Chioncel, John G F Cleland, Andrew J S Coats, Maria G Crespo-Leiro, Dimitrios Farmakis, Martine Gilard, Stephane Heymans, Arno W Hoes, Tiny Jaarsma, Ewa A Jankowska, Mitja Lainscak, Carolyn S P Lam, Alexander R Lyon, John J V McMurray, Alexandre Mebazaa, Richard Mindham, Claudio Muneretto, Massimo Francesco Piepoli, Susanna Price, Giuseppe M C Rosano, Frank Ruschitzka, Anne Kathrine Skibelund, and ESC Scientific Document Group. 2021 ESC guidelines for the diagnosis and treatment of acute and chronic heart failure: developed by the Task Force for the diagnosis and treatment of acute and chronic heart failure of the European Society of Cardiology (ESC) with the special contribution of the Heart Failure Association (HFA) of the ESC. *European Heart Journal*, 42(36):3599–3726, 08 2021. ISSN 0195-668X. doi: 10.1093/eurheartj/ehab368. URL https://doi.org/10.1093/eurheartj/ehab368.

[2] NHS South Tees Hospitals. Sodium (and urine), 2025. URL https://www.southtees.nhs.uk/services/pathology/tests/sodium-and-urine. Last accessed 08 November 2025.

[3] Tanvir Ahmad, Assia Munir, Sajjad Haider Bhatti, Muhammad Aftab, and Muhammad Ali Raza. Survival analysis of heart failure patients: A case study. *PLOS ONE*, 12(7):1–8, 07 2017. doi: 10.1371/journal.pone.0181001. URL https://doi.org/10.1371/journal.pone.0181001.

[4] NHS Gloucestershire Hospitals. Haematology reference ranges, 2025. URL https://www.gloshospitals.nhs.uk/our-services/services-we-offer/pathology/haematology/haematology-reference-ranges/. Last accessed 08 November 2025.

[5] Max Kunn. caret (version 7.0-1): createDataPartition: Data Splitting functions, 2025. URL https://www.rdocumentation.org/packages/caret/versions/7.0-1/topics/createDataPartition. Last accessed 09 November 2025.

[6] Trevor Hastie, Robert Tibshirani, Jerome Friedman, and James Franklin. The Elements of Statistical Learning: Data Mining, Inference and Prediction. *The Mathematical Intelligencer*, 27(2):83–85, 2005.

[7] American Heart Association. Smoking and High Blood Pressure, 2024. URL https://www.heart.org/en/health-topics/high-blood-pressure/changes-you-can-make-to-manage-high-blood-pressure/smoking-high-blood-pressure-and-your-health. Last accessed 09 November 2025.

[8] John Hopkins Medicine. Diabetes and High Blood Pressure, 2025. URL https://www.hopkinsmedicine.org/health/conditions-and-diseases/diabetes/diabetes-and-high-blood-pressure. Last accessed 09 November 2025.

[9] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006. ISSN 0167-8655. doi: https://doi.org/10.1016/j.patrec.2005.10.010. URL https://www.sciencedirect.com/science/article/pii/S016786550500303X.

[10] Şeref Kerem Çorbacıoğlu and Gökhan Aksel. Receiver operating characteristic curve analysis in diagnostic accuracy studies: A guide to interpreting the area under the curve value. *Turkish Journal of Emergency Medicine*, 2023. doi: 10.4103/tjem.tjem_182_23.

[11] J A Hanley and B J McNeil. A method of comparing the areas under receiver operating characteristic curves derived from the same cases. *Radiology*, 148(3):839–843, 1983. doi: 10.1148/radiology.148.3.6878708.

[12] Mary L McHugh. Interrater reliability: the kappa statistic. *Biochemia medica*, 2012. doi: 10.11613/bm.2012.031.

[13] Davide Chicco. Ten quick tips for machine learning in computational biology. *BioData Mining*, 2017. doi: 10.1186/s13040-017-0155-3.

Appendix

Appendix B1: Histograms of continuous variables

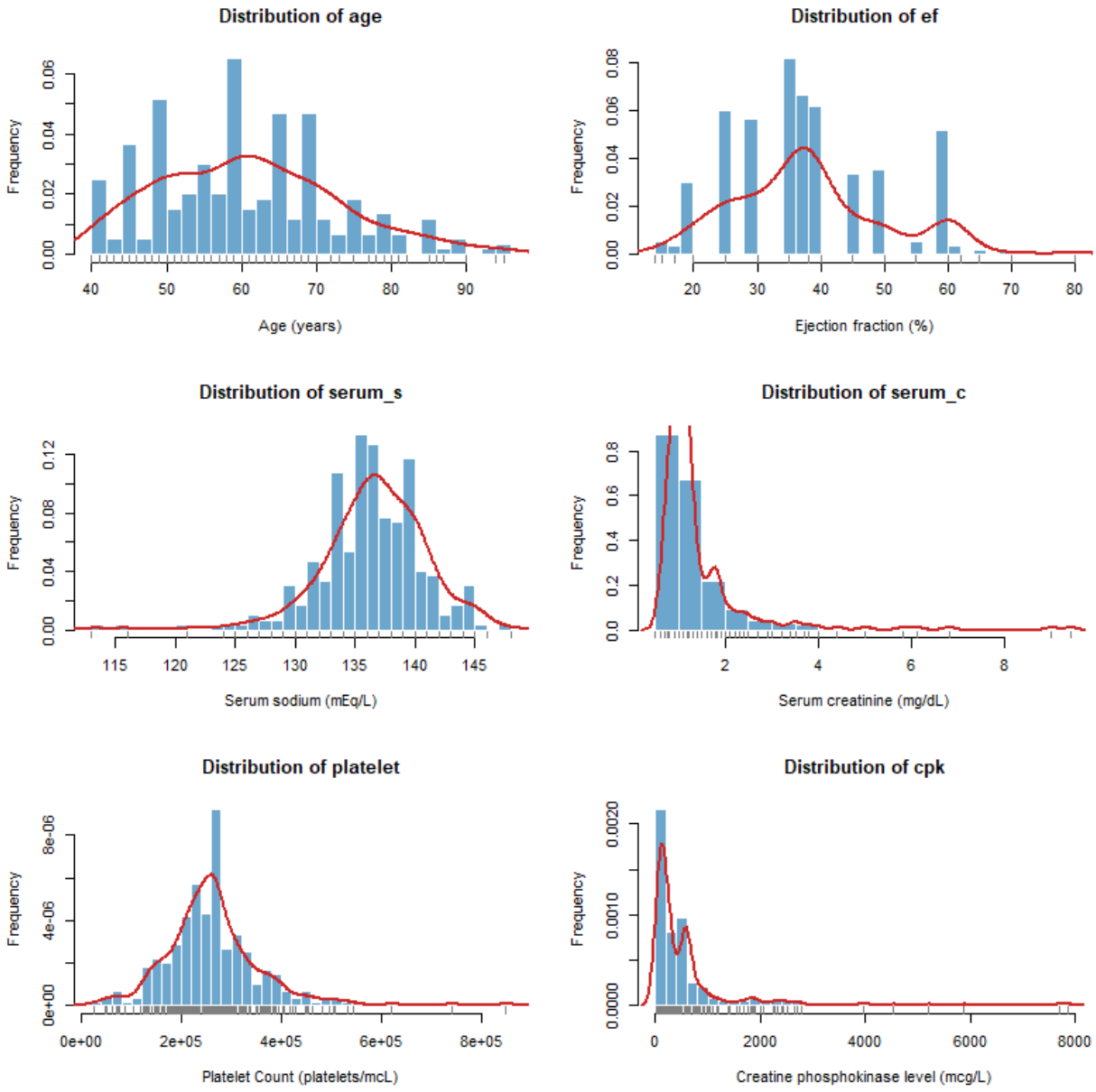

Figure 1: Histograms of continuous variables with density overlays showing the distribution shapes.

Appendix B2: Boxplots of continuous variables

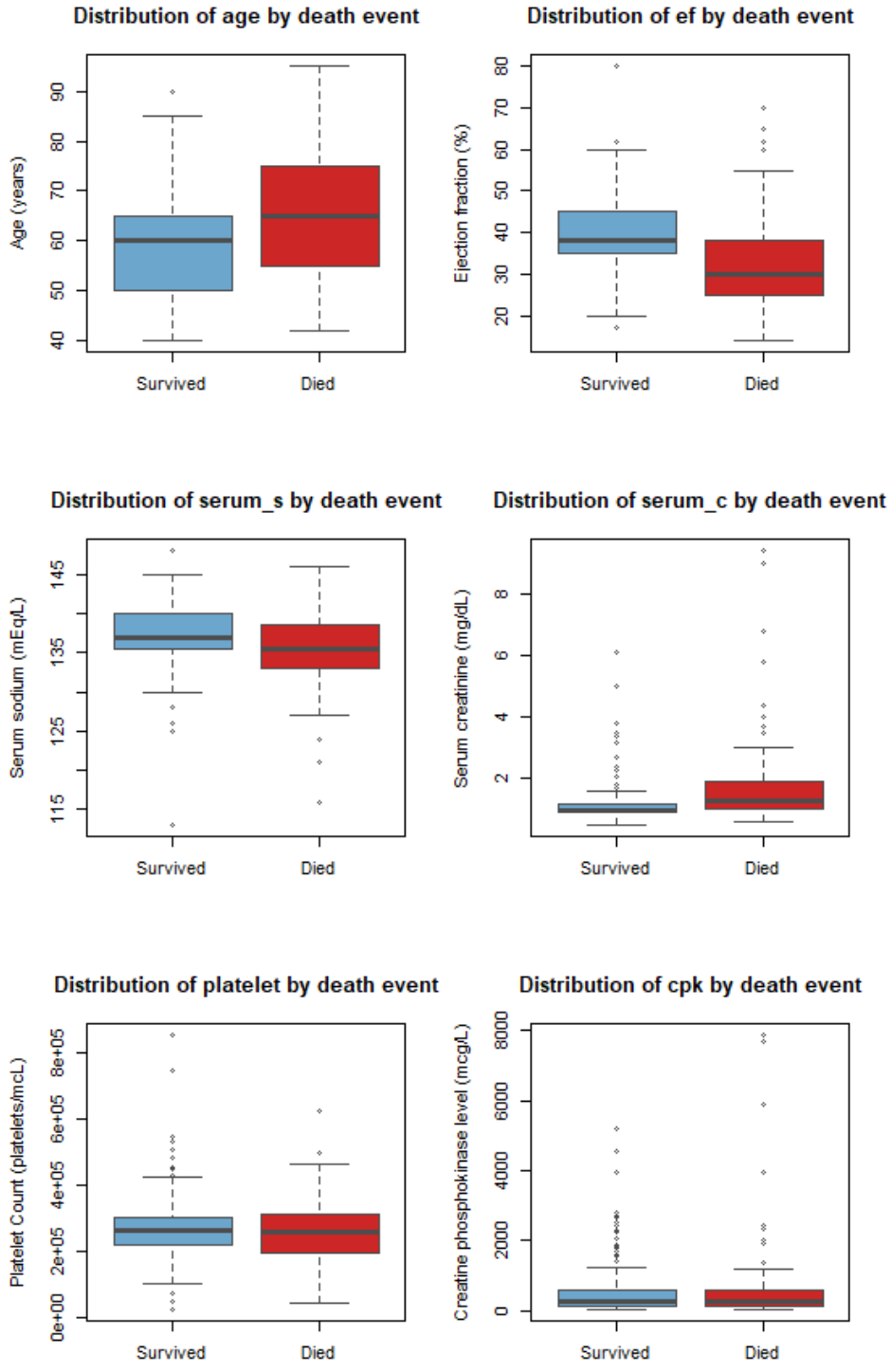

Figure 2: Boxplots of categorical variables based on death outcome.

Appendix B3: Barplots of categorical variables

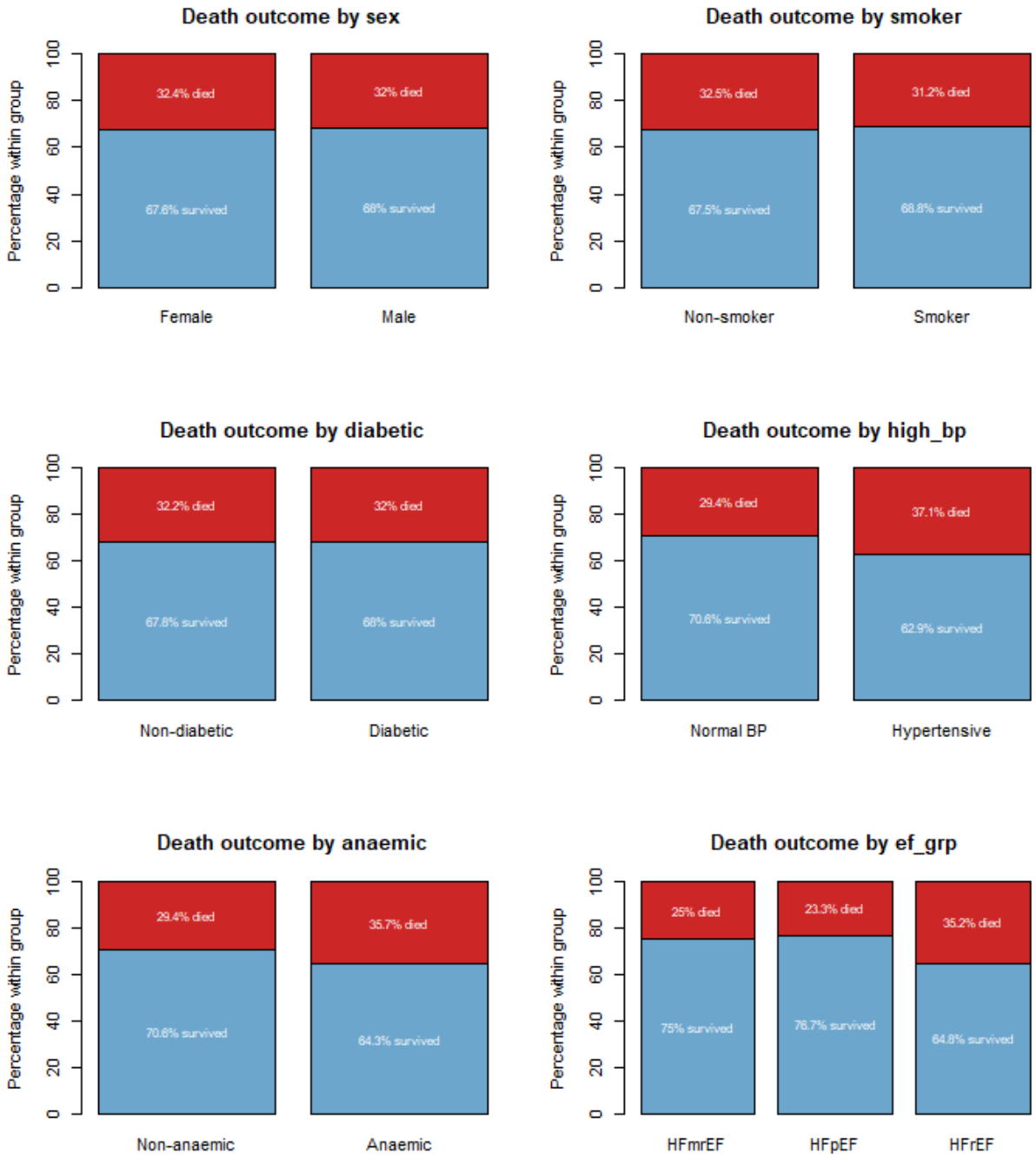

Figure 3: Barplots of categorical variables based on death outcome.

Appendix D1: Residual plots of continuous predictors

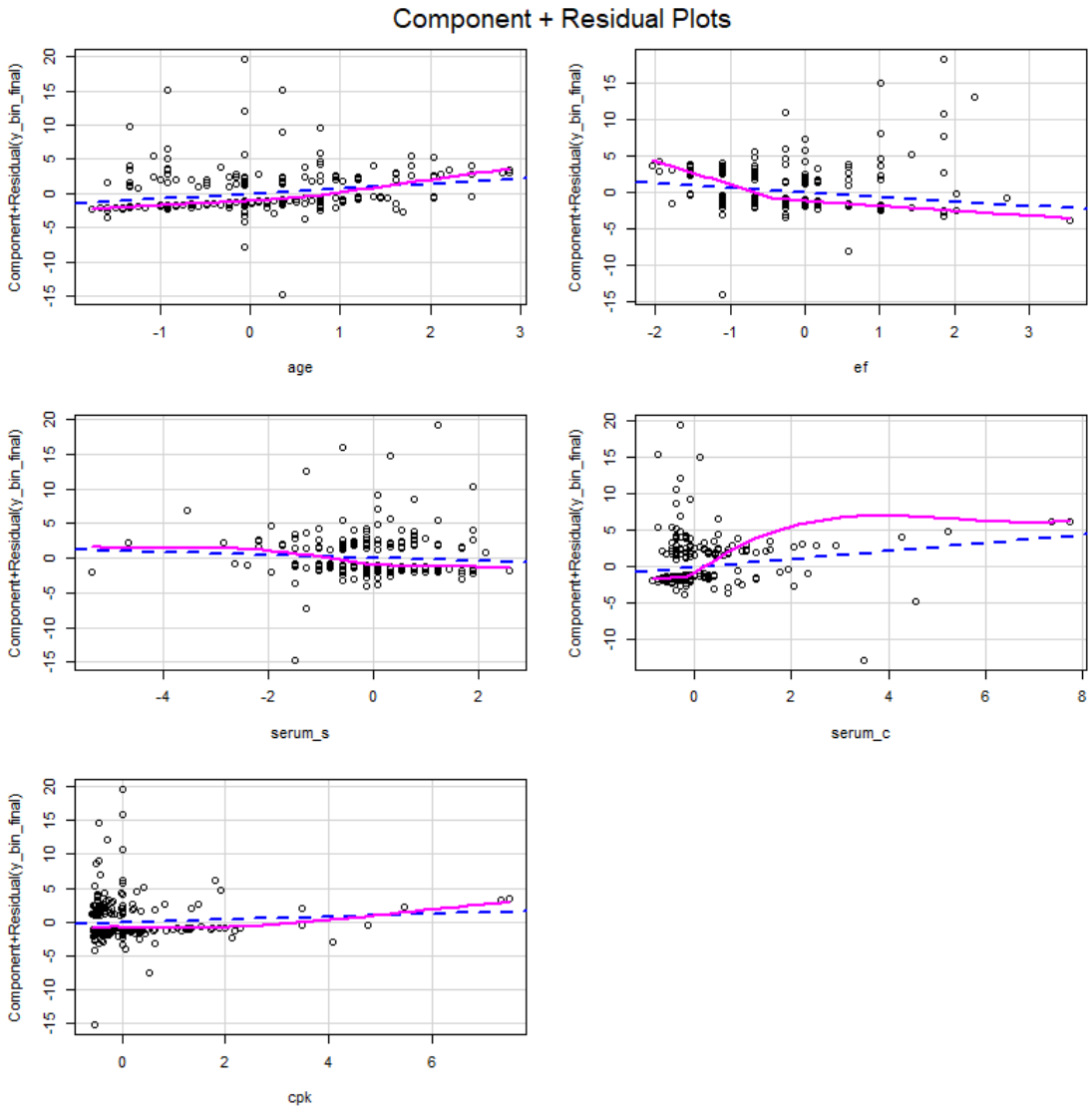

Figure 4: Component-plus-residual (CR) plots of each significant continuous predictor, based on the weighted logistic regression model.

Appendix D2: Binned residual plot of weighted logistic regression model

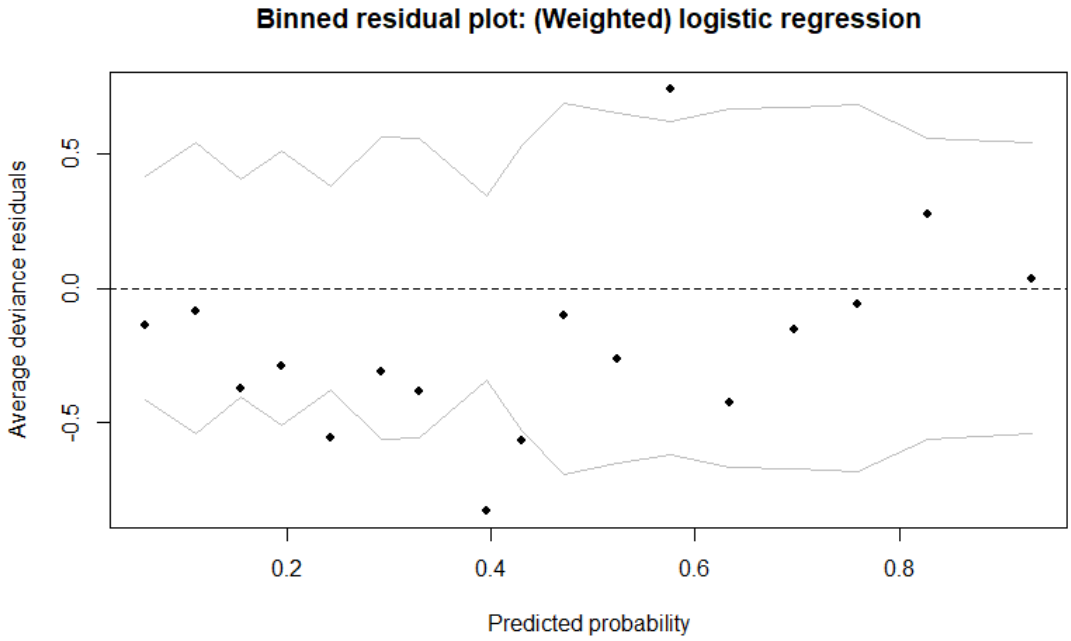

Figure 5: Binned deviance residual plot of the weighted logistic regression model.

Appendix D3: ROC plot of weighted logistic regression model

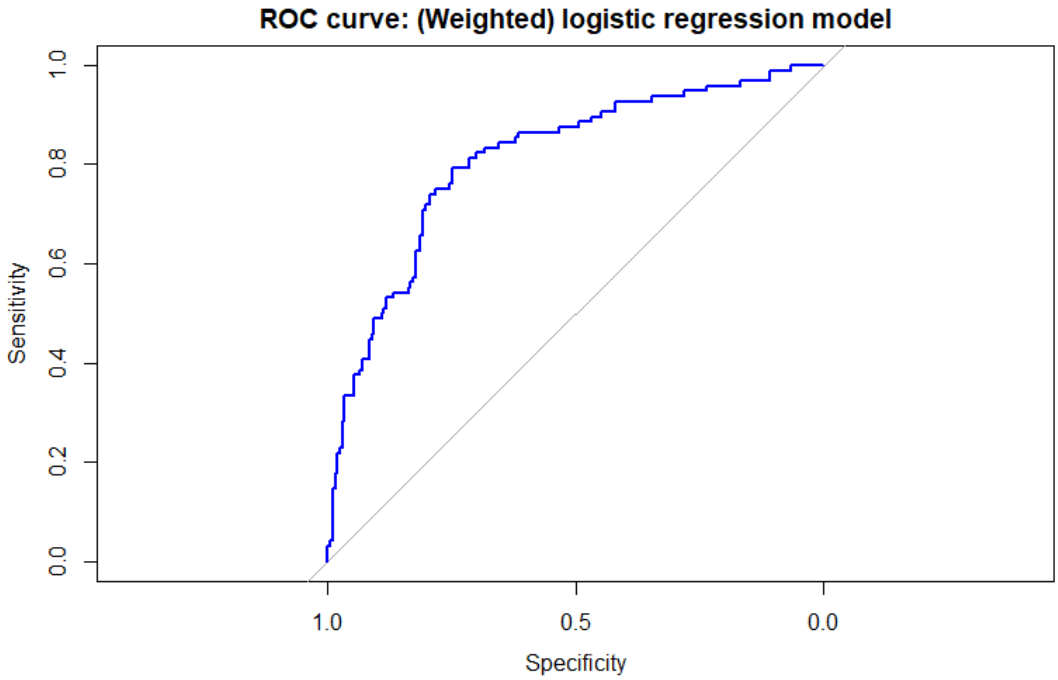

Figure 6: Receiving Operator Characteristic (ROC) curve of the weighted logistic regression model.

Appendix D4: Calibration plot of weighted logistic regression model

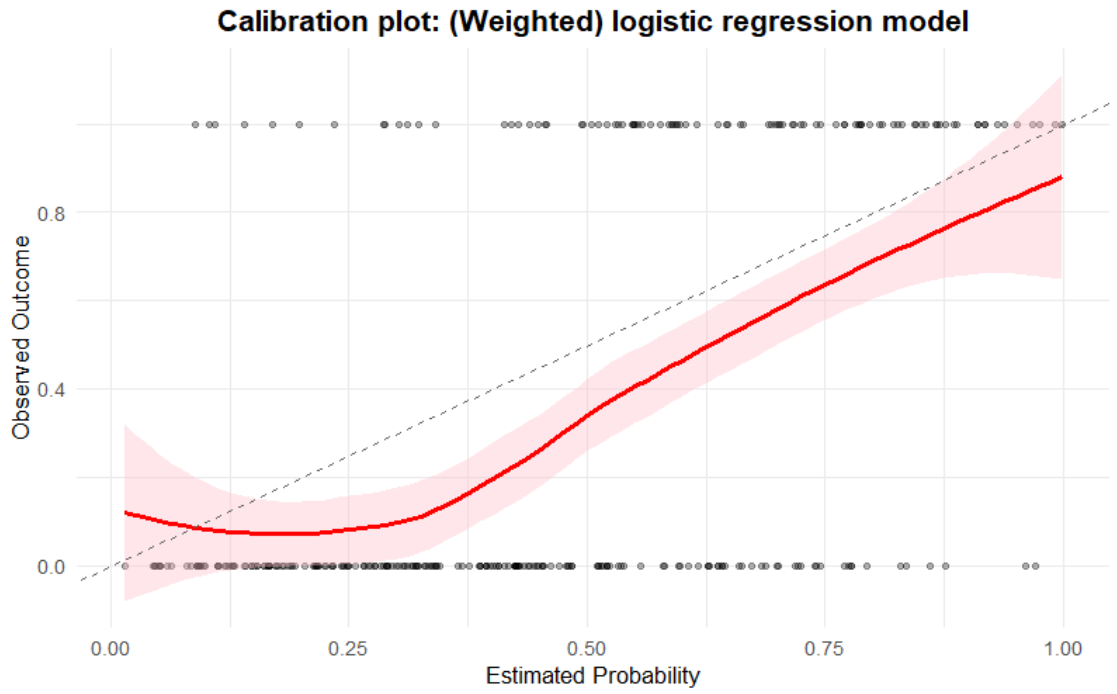

Figure 7: Calibration plot of the weighted logistic regression model with 95% confidence intervals.

Appendix D5: Natural cubic spline plots of continuous predictors

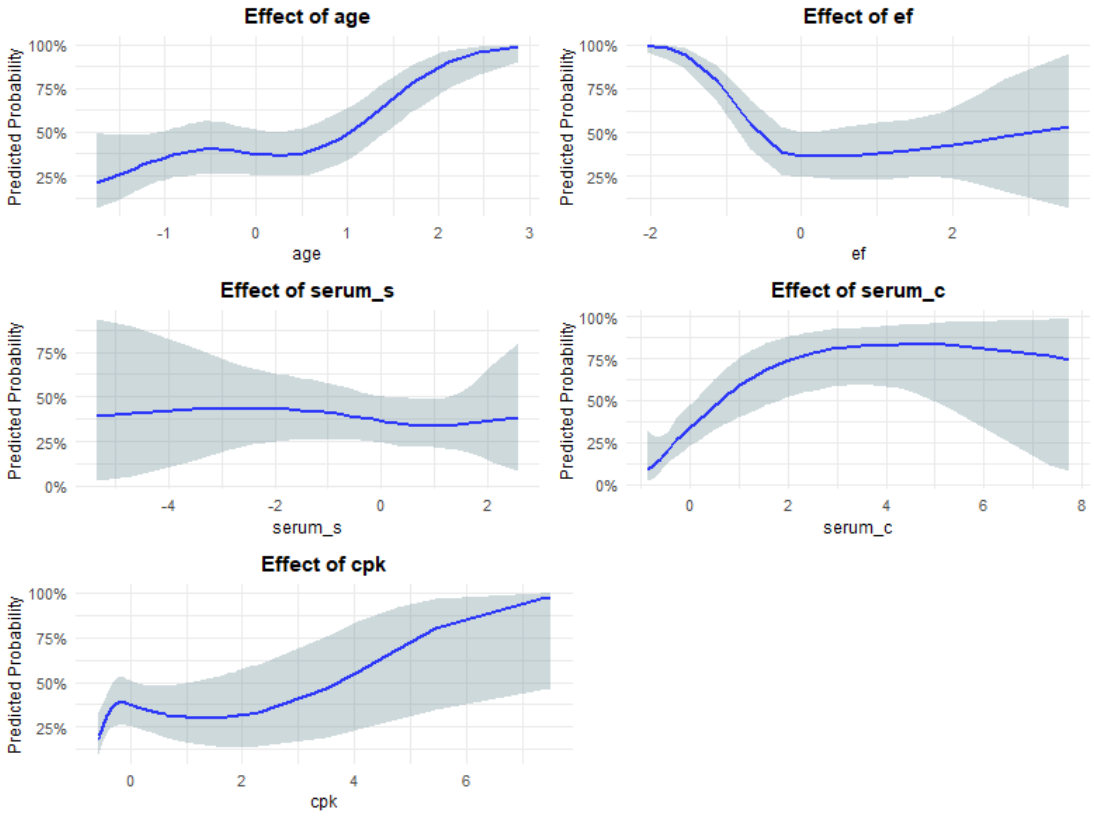

Figure 8: Natural cubic spline plots of each significant continuous predictor (based on the weighted logistic regression model), with 95% confidence intervals

Appendix D6: Calibration plot of weighted cubic spline model

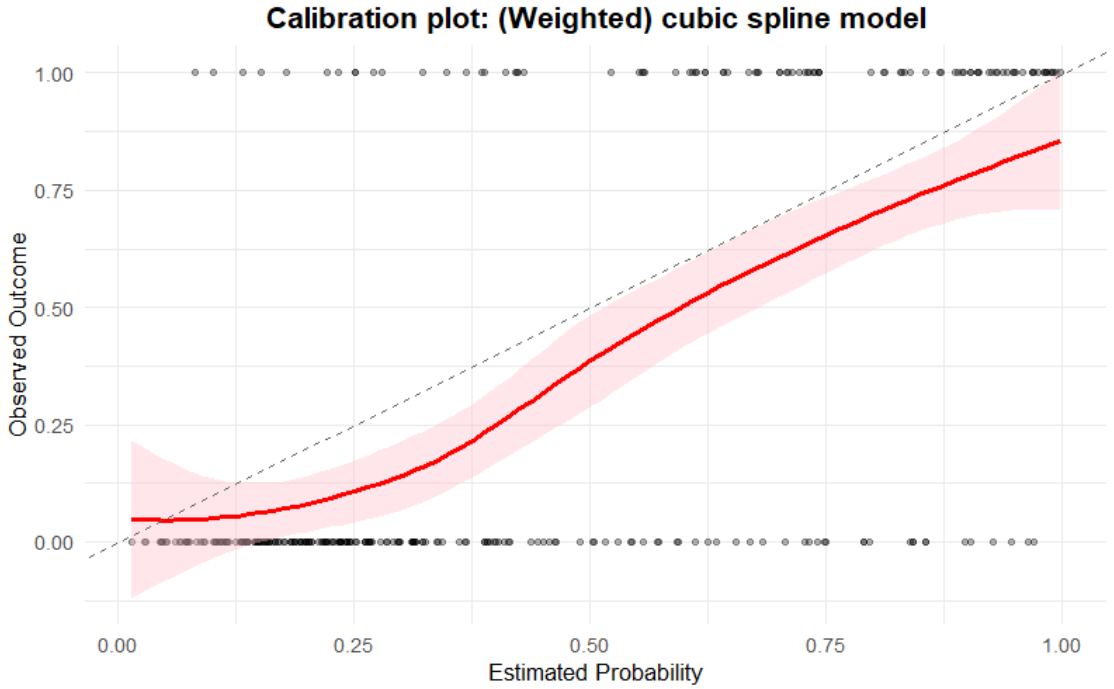

Figure 9: Calibration plot of the weighted natural cubic spline model, with 95% confidence intervals.